Thomas Nguyen
11888002
Professor Zhe Dang
Due 5/1/26

Problem 1:

For any three NP-problems A, B, C, if $A \leq_m B$ and $B \leq_m C$, then $A \leq_m C$.

Proof:

Since $A \leq_m B$, there exists a polynomial-time computable function f such that for all x:

$$x \in A \iff f(x) \in B$$

Since $B \leq_m C$, there exists a polynomial-time computable function g such that for all y:

$$y \in B \iff g(y) \in C$$

Define $h(x) = g(f(x))$.

Since f and g are both computable, their composition $h = g \circ f$ is also computable.

h runs in polynomial time: Suppose f runs in time $O(n^j)$ and g runs in time $O(m^k)$. On input x of length n, computing $f(x)$ takes $O(n^j)$ time and $|f(x)| \leq O(n^j)$. Then computing $g(f(x))$ takes $O((n^j)^k) = O(n^{jk})$ time. Total time is $O(n^{jk})$, which is polynomial.

h satisfies the reduction property: Take any x.

Forward: If $x \in A$, then $f(x) \in B$ by reduction 1. Then $g(f(x)) \in C$ by reduction 2. So $h(x) \in C$.

Backward: If $h(x) = g(f(x)) \in C$, then $f(x) \in B$ by reduction 2. Then $x \in A$ by reduction 1.

Thus h is a polynomial-time many-one reduction from A to C, so $A \leq_m C$.

Problem 2

Given a directed graph G, is there a path on G such that every node is covered exactly once?

This is the Hamiltonian Path problem. We need a nondeterministic polynomial-time algorithm.

Let $G = (V, E)$ be a directed graph with $n = |V|$ nodes. Let $p = (v_1, v_2, \ldots, v_n)$ be a candidate path. Let visited be a boolean array of size n, initialized to False.

Guess: Nondeterministically guess a sequence $p = (v_1, v_2, \ldots, v_n)$ containing all n nodes.

Check each consecutive pair in p: For each pair (v_i, v_{i+1}), if $\text{visited}[v_i]$ is False, set it to True. If it is already True, return False (node repeated). If edge $(v_i, v_{i+1}) \notin E$, return False.

Check coverage: If any entry in visited is still False, return False. Otherwise return True.

Nondeterministically generates a permutation of n nodes. Iterates through $n-1$ pairs, each edge lookup is $O(1)$ with an adjacency matrix, so $O(n)$ total. Scans the array in $O(n)$. Total verification time is $O(n)$, which is polynomial.

The problem is in NP because a nondeterministic Turing machine can guess a certificate and verify it in polynomial time.

Problem 3

Given a directed graph G, is there a path on G such that every node is covered (visited at least once, repeats allowed)?

The difference from Problem 2 is that nodes can be revisited. We just need every node to appear somewhere on the path.

Let $G = (V, E)$ with $n = |V|$. The certificate is a path $p = (v_1, v_2, \ldots, v_k)$ where $k \leq n^2$. The path length must be polynomials bounded for this to be in NP, and if a covering path exists, one exists with length at most n^2.

Guess: Nondeterministically guess a path $p = (v_1, v_2, \ldots, v_k)$ where $k \leq n^2$.

Verify edges: For each consecutive pair (v_i, v_{i+1}), if $(v_i, v_{i+1}) \notin E$, return False.

Check coverage: For each node $u \in V$, if u does not appear anywhere in p, return False.

If all checks pass, return True.

checks at most n^2 edges. checks n nodes against the path, which takes $O(n^2)$ with a hash set marking nodes seen in p. Total is polynomial.

The problem is in NP.

Problem 4

Given: A deterministic polynomial-time algorithm $\text{SAT-DECIDE}(C)$ that decides whether a Boolean circuit C is satisfiable. Two Boolean circuits C_1 and C_2, each with inputs $(x_1, \ldots, x_n)$ and one output y.

C_1 and C_2 are equivalent if for every input:

$$C_1(x_1, \ldots, x_n) = C_2(x_1, \ldots, x_n)$$

Goal: Show there exists a deterministic polynomial-time algorithm to decide equivalence.

C_1 and C_2 are not equivalent if and only if there exists some input where their outputs differ. We can detect this using $\mathrm{SAT\text{-}DECIDE}$.

Construct a new circuit C_{xor}:

$$C_{xor}(x_1, \ldots, x_n) = (C_1 \wedge \neg C_2) \vee (\neg C_1 \wedge C_2)$$

This is the XOR of the two circuits. It outputs 1 if and only if C_1 and C_2 produce different outputs on that input.

The size of $C_\oplus$ is $|C_1| + |C_2| + O(1)$ since it reuses C_1 and C_2 as sub-circuits and adds only a constant number of extra gates (two NOT, two AND, one OR). Construction time is polynomial.

The algorithm:

```
EQUIV-DECIDE(C_1, C_2):
  1. Build C_xor = (C_1 AND NOT C_2) OR (NOT C_1 AND C_2)
  2. result = SAT-DECIDE(C_xor)
  3. If result == "satisfiable": return "NOT equivalent"
     Else: return "Equivalent"
```

Correctness: $C_\oplus$ is satisfiable iff there exists an input where C_1 and C_2 differ, which means they are not equivalent. C_{xor} is unsatisfiable iff no such input exists, meaning $C_1 \equiv C_2$.

Runtime: Building C_{xor} takes $O(|C_1| + |C_2|)$. Running $\mathrm{SAT\text{-}DECIDE}$ is polynomial by assumption. Total is polynomial